Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Camila Rios	camirios@udel.edu
Friend (optional)	Tifany Sonnenberg	tsonnen1@kent.edu

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea and a paragraph describing how that paper supports your design concept.

        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider creating an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the ChemPy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages from uploaded images.
    - **Mechanical Engineering** majors may consider a project that somehow uses both the NumPy and SciPy Python libraries.

The goal for this website is to create a library of alternative music that recommends a user a song depending on a mood or theme. The library will only have some songs, so a user could submit a suggestion for a song to add to the library.

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

Websites like Chosic use AI to allow users to find songs based on a theme, websites like Music Mood Finder actually allow the user to input an artist and/or song and find the mood or genre associated with the input.

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

The legitimate purpose of this website is to allow users to expand their personal music library in a fast and efficient mode. I know that when curating my playlists I wanted to find

songs that fit a specific mood or theme, but finding them was always hard. I want this website to make that process easier.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

The ultimate goal is recommending a user a song that fits the mood they are seeking.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

A mood or theme that could be associated with a song, ex) Romantic, Moody, Dark

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

Songs will be stored as classes, one of their attributes will be a list of strings that represent moods or themes. The program will match the input to one of those strings and return the song.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

The user will get back a song recommendation, and below an option to recommend their own song to be added to the library.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

The core things the program needs to track are: Theme, Mood, Song, Theme

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

For Song *Details I need: Song Name, Artist, Theme* *Details I will Ignore: Language, Duration*

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

I think the most difficult or technically complex thing will be storing the recommendations so the developer (me) can see them. Considering multiple songs may have the same mood or themes, randomizing the answer may be hard too.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

I will probably be using an LLM to help, what I am going to do is, first try to code parts by myself, then ask for help when I am stick and urge the LLM to give me hints rather than code, and if it does give me code I will ask it to explain every step to me.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

I hope to build something people can actually use and I can have fun with! While also getting a good grade.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

My Prompt: I want to make a simple website, with python, that allows a person to enter a mood a theme and recommends them as song. I also want to add an option where they can recommend a song to add to the library. What are some tools or functions the website could have? Do you think it would be a good idea to also show the image of the album of the song returned?

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

What Gemini Responded (summarized): Gemini agreed that adding the cover art to the result page would be a good and fun feature. It suggested that the input have either a dropdown or tag function so its easier for the user to input the theme and/or mood they are looking for. It would also make the matching easier. Gemini also suggested a surprise me button that allows the user to just get a random song with no input on their part.

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

My response to Gemini: This is for a beginner first year computer science course. While the database, I have not exactly learnt how to handle them, would definitely be useful. Is there an easier way to store the songs in the "library" ? Maybe they could be stored in a list of classes *?*

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

I had originally planned to make a feature so that the user could see the full library but decided that it would be too complicated, especially since it would most likely need a database. Through this though, I definitely have a more concrete idea on how I want the song recommendations to work and I will also make the pages have more visual elements than just text.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

If I get stuck, I will primarily go to office hours, I have also previously watched videos on python and will most likely continue to do so, and I could also use the AI Tool provided by the class to guide it in helping me.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

I will try to revise the code myself and maybe compare it to the code from past projects to see if there are any glaring red flags. Again, I could also go to office hours and ask for help if I am really stuck. Once the issue is identified I will use the skills I have learned to fix it!

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

For the first part of the project I asked my friend, Tifany, for help and she's the one that helped me come up with the recommendation system, she had recommended a website that recommended albums that were similar to the album the user imputed, I liked the idea and tweaked it.

The only part I used an LLM for was part 5, here is the full conversation: (it would not let me copy paste the text for some reason:

I want to make a simple website, with python, that allows a person to enter a mood a theme and recommends them as song. I also want to add an option where they can recommend a song to add to the library. What are some tools or functions the website could have? Do you think it would be a good idea to also show the image of the album of the song returned?

Building a mood-and-theme song recommender is a fantastic project! It combines basic web development, logic/filtering, and data storage.

Here is a breakdown of the tools and functions you could use to bring this to life, along with thoughts on incorporating album art.

Recommended Python Tools & Tech Stack

To build a web app entirely (or mostly) in Python, you have a couple of great options depending on how you want to handle the interface:

- **Streamlit (Easiest & Fastest):** If you want to build a web app using *only* Python without messing with HTML, CSS, or JavaScript, Streamlit is incredible. It handles input fields, buttons, data display, and images out of the box with just a few lines of code.
- **Flask (More Customizable):** If you want to learn traditional web development, Flask lets you write Python for the backend and HTML/CSS for the frontend.
- **Database (SQLite or JSON):** You'll need a way to store your song library and the user-submitted recommendations. For a simple app, a local JSON file or a lightweight SQLite database (easily managed via Python's built-in `sqlite3` module) works perfectly.
- **Spotify Web API (Optional):** If you want real-world song data, previews, and album covers without manually curating a library, you can connect your app to the Spotify API using the `spotipy` Python library.

Core Features & Functions for the Website

1. **The Recommendation Engine (Input & Match):**
    - **Inputs:** Dropdowns or text boxes where users select or type their current **mood** (e.g., *Energetic, Melancholic, Relaxed*) and **theme** (e.g., *Late-night drive, Studying, Rainy day*).
    - **Matching Logic:** A function that fi[illegible] your song library based on tags matching both the mood and theme to retur[illegible]curated song suggestion.

2. **The Song Submission Portal:**
    - **Inputs:** Form fields where users can input a song title, artist, mood tag, theme tag, and a link (or album cover URL).
    - **Moderation/Storage Function:** A function that appends these new suggestions to your database or JSON file so they immediately (or after a quick review) join the library pool.
3. **Randomize / "Surprise Me" Button:**
    - A fallback function that picks a totally random song from the library if the user can't decide on a mood or theme.

Should You Show the Album Artwork?

Yes, absolutely! Showing the album art is one of the best choices you can make for this type of app.

- **Visual Engagement:** Music is a deeply sensory experience. Plain text titles look dry, but a vibrant album cover instantly grabs attention and makes the recommendation feel real and professional.
- **Instant Recognition:** Users often recognize a song faster by its album art than by reading the title or artist name.
- **How to implement it:** If you use a hardcoded library, you can store image URLs alongside the song data. If you use the Spotify API, the album artwork URL is automatically included in the track search results, making it very easy to display using Streamlit's `st.image()` or an HTML `<img>` tag in Flask.

Want me to write a basic prototype of this app using Streamlit?

↪ Yes

This is for a beginner first year computer science course. While the database, I have not exactly learnt how to handle them, would definitely be useful is there an easier way to store the songs in the "library" ? Maybe they could be stores in a list of classes?

Yes, absolutely! For a first-year computer science project, you do not need a database at all. Storing your songs directly in Python memory using built-in data structures or custom classes is standard practice, very easy to set up, and a great way to practice your programming fundamentals.

Here are the two best ways to do it without a database:

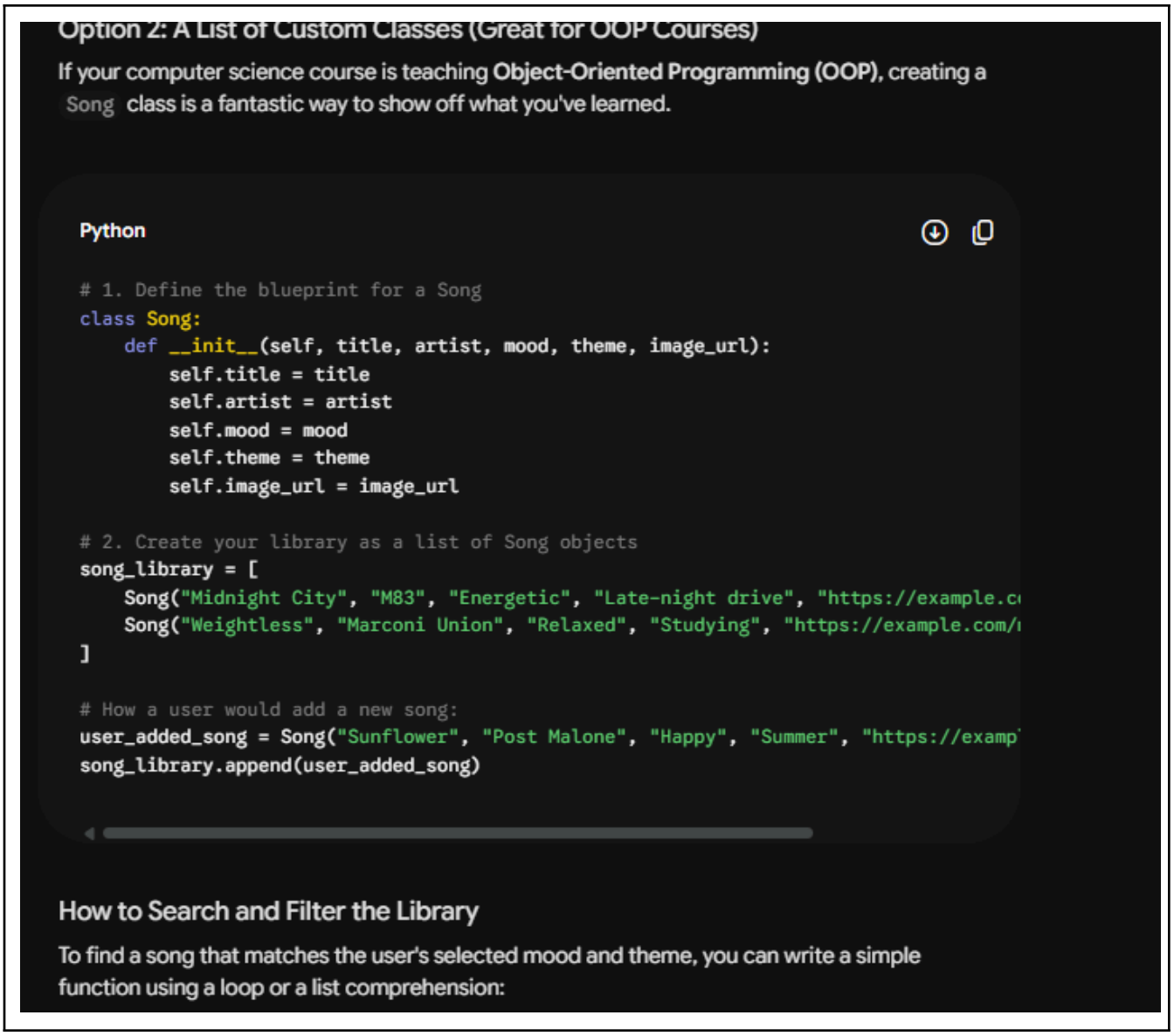
Option 1: A List of Dictionaries (The Easiest Way)

Dictionaries are great for pairing keys (like `"title"`, `"mood"`, etc.) with values. You can put multiple dictionaries inside a standard Python list.

Python

```python
# Your starting song library
song_library = [
    {
        "title": "Midnight City",
        "artist": "M83",
        "mood": "Energetic",
        "theme": "Late-night drive",
        "image": "https://example.com/m83.jpg"
    },
    {
        "title": "Weightless",
        "artist": "Marconi Union",
        "mood": "Relaxed",
        "theme": "Studying",
        "image": "https://example.com/marconi.jpg"
    }
]

# How a user would add a new song:
new_song = {
    "title": "Sunflower",
    "artist": "Post Malone",
    "mood": "Happy",
    "theme": "Summer",
    "image": "https://example.com/sunflower.jpg"
}
song_library.append(new_song)
```



Option 2: A List of Custom Classes (Great for OOP Courses)

If your computer science course is teaching **Object-Oriented Programming (OOP)**, creating a `Song` class is a fantastic way to show off what you've learned.

Python

```
# 1. Define the blueprint for a Song
class Song:
    def __init__(self, title, artist, mood, theme, image_url):
        self.title = title
        self.artist = artist
        self.mood = mood
        self.theme = theme
        self.image_url = image_url

# 2. Create your library as a list of Song objects
song_library = [
    Song("Midnight City", "M83", "Energetic", "Late-night drive", "https://example.c
    Song("Weightless", "Marconi Union", "Relaxed", "Studying", "https://example.com/
]

# How a user would add a new song:
user_added_song = Song("Sunflower", "Post Malone", "Happy", "Summer", "https://examp
song_library.append(user_added_song)
```

How to Search and Filter the Library

To find a song that matches the user's selected mood and theme, you can write a simple function using a loop or a list comprehension:

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review [this page of the Drafter documentation] to better prepare for the next step.